SOUP INTERNSHIP · TAKE-HOME TASK

Full-Stack Engineer

Work: 5–7 h · Deadline: 48 h · Any stack · Run: docker compose up

TASK

A mini experiment tracker, and answers to two questions: does what the user sees always match what is stored, and do they see only what's theirs?

REQUIREMENTS

Projects and API keys. Metric ingestion via POST /runs/{id}/metrics in batches from a client on an unreliable network. Storage. A run web page that updates live (SSE or WebSocket). A run list.

1. Design before code. One page: event format and ordering, what happens when the same batch is sent again, how a client catches up on missed events after a disconnect, how projects are isolated.

2. Prove consistency. An end-to-end test (UI and API): metrics arrive out of order, duplicated and late, and the connection drops and recovers mid-stream. The final page state must match the data in the database. Include the output and state what the test would NOT catch.

3. Prove isolation. Automated tests: project A can't read, write, subscribe to the live stream of, or guess the IDs of project B (including via SSE or WebSocket and via cache). What else could leak: logs, error messages, timing?

4. Silent failures. Events lost on reconnect, duplicates, queue overflow for a slow client, partial writes, an old API key that keeps working after rotation. How you checked each.

5. Verdict. SHIP or DON'T SHIP for 50 concurrent runs at 10 events per second, and what to change first.

DELIVERABLES

Repository and docker compose · tests and raw output · a note of ≤2 pages · one paragraph on what surprised you and what still concerns you. Submit by email reply to team@trysoup.dev. Don't edit or hide failed results: a clear negative beats a quiet success. If something breaks, tell us what happened, what you tried, and what you learned.

AI TOOLS

Allowed (we use them too). Note where you used them, which suggestions you accepted, and what you verified, changed, or redid yourself. Not a penalty: we want to see how you work and where you spot an unreliable tool.

LIVE DISCUSSION

40 minutes, after submission. Don't pre-write answers; we want to think it through with you. Possible questions: A client was offline for 30 seconds: what does it see and why? Where is your design weakest? How would you know in production that an event was lost?

SCORING

Consistency and isolation — 30% · silent failures — 20% · architecture and code quality — 15% · live discussion — 35%. We're not just checking whether a metric improved. We want an engineer who questions assumptions, checks evidence, and understands what the system is actually doing.

Good luck. Be precise, honest, and curious.